Smart Knowledge Base for Agents — Architecture & Design Decisions

1. Idea in one paragraph

Plain retrieval fails in two ways the brief names: near-duplicate documents (same topic, different year / version / entity) are confused, and answers spread over many pages are only partly retrieved. Our engine treats retrieval as **navigation before similarity**. At ingestion it reads the unorganised dump and builds, per document, a **card** (entity, type, period, status, …), **"how to tell apart" notes** for look-alike documents, and a **page-bounded section tree**. At query time an LLM first **routes** the question to the right documents using those cards; search then runs only **inside** those documents, once per checklist item; a fast LLM **grades** every candidate passage; and a single **answer** call writes a cited answer from the directly relevant passages only, or abstains.

2. Ingestion — `POST /index` (`kb/parse.py`, `kb/index.py`)

Step	What it does	Why
Parse	PyMuPDF (pinned 1.26.5) in a crash-safe process pool; ruled tables → markdown; Tesseract OCR for pages without a text layer; ligature / number normalisation; running headers and footers removed	One training PDF is fully scanned; MuPDF can segfault on odd pages, so a dead worker never stalls ingestion
Structural units	Text split along headings, **never across a page**, ~250–2,200 characters; each unit attached to a section tree built from the PDF outline or detected headings	Citations need document + page; sections give the "whole topic" a question may need
Document cards	1 LLM call per document: title, entity, issuer, type, period, date, jurisdiction, status, topics, summary	File names are unreliable (renamed files, register numbers, scanner names); identity must come from content
Look-alike notes	MinHash text overlap or same entity + type → an LLM writes how the two documents differ	Directly targets failure mode 1 (near-duplicates)
Contextual headers + index	Each unit prefixed `[document | section path | page]`, embedded with MiniLM (local, no per-query cost) + BM25	Headers carry year / entity into search; local embeddings keep query cost at LLM calls only

Ingestion of the training corpus (36 PDFs, 7,091 pages, 24,013 units): **≈ 15 minutes cold** on 8 cores (9 min parsing incl. OCR, ~6 min cards + embeddings, 48 LLM calls ≈ $0.05). Parse and embedding caches make re-indexing unchanged files near-instant. Multiple named collections are kept side by side; a collection is built in a temporary folder and swapped in only when complete.

3. Query — `POST /query` (`kb/query.py`): route → retrieve → grade → answer

#	Step	LLM	What it does
1	`route`	1 call (Haiku 5.5)	Reads the whole card catalogue: picks documents (matching entity / period / version, preferring a specific document over a compendium), classifies the question (lookup / analysis / comparison), writes a checklist and search queries, flags documents the question needs but the collection lacks
2	`retrieve`	—	Dense + BM25 search **restricted to the routed documents**, one search per query and checklist item (merged by best score); hits expanded to their small leaf section; near-duplicates collapsed; round-robin budget across documents so one large document cannot crowd out others; falls back to the whole collection if routing is empty or unsure
3	`grade`	parallel Haiku calls	Every candidate graded 2 = directly answers / 1 = needed context / 0 = not needed (topic-only, boilerplate, wrong entity / period / version)
4	`answer`	1 call (Haiku 5.5)	Written only from grade-2 passages, bounded length (lookup ≤ 150 words, analysis ≤ 600, comparison ≤ 650), every claim cited `[file pN]`, explicit "the documents do not state …" for gaps and for missing documents; **abstains with no chunks** when nothing is directly relevant

The response carries `answer`, `chunks` (document, page, section, grade, text) and `trace` (one `{step_name, description, input, output}` per step); every step and LLM call is also a Langfuse span / generation with tokens and cost.

How it addresses the two failure modes. Near-duplicates are resolved at the *document* level (cards + look-alike notes) before similarity search runs, so similar text from the wrong year or entity never competes. The chunk-size dilemma is handled by searching per checklist item, expanding hits to their sections, and letting the grader decide relevance passage by passage, instead of hoping a larger top-k happens to contain everything.

4. Results (training set, dev judge)

The supplied evaluator's configured judge (a reasoning model) exhausts its 10,000-token budget on long answers and returns empty JSON, so we self-evaluated with the **notebook code unmodified** but its requests answered by Claude Haiku 5.5 (thinking disabled). 40 questions: 20 used during development + 20 held-out never used for tuning.

Set	Answer P	Answer R	Context P	Context R	Latency / query	Cost / query
Development 20	0.32	0.62	0.92	0.57	24 s	$0.017
Held-out 20	0.37	0.56	0.86	0.54	22 s	$0.016
Judge ceiling (gold answers / chunks scored as ours)	≈ 0.95	≈ 0.97	≈ 0.78	≈ 0.78		

Context precision exceeds what the judge awards the gold chunks themselves; context recall and the answer metrics are the remaining gap. Cost: ≈ 145k input + 9k output Haiku tokens per query.

5. What we tried and dropped (measured on the same questions)

Idea	Effect	Decision
LLM section selection over the document tree (PageIndex-style), gap-fill round, editor pass, primary/supporting weighting (pipeline v1, 9 steps)	Same scores within noise as the 4-step pipeline, ~21% slower, ~14% more expensive	Removed
Claim bank: extract atomic claims, select per checklist item, write facet by facet	Answer recall ↑ (0.65) but precision ↓ (0.20–0.25): answers carried more points than the gold	Dropped
Returning grade-1 "context" passages as chunks	Context recall ↑ (0.80) but precision ↓ (0.63)	Only when grade-2 evidence is thin
deepseek-v4.1-flash for the pipeline	Provider stopped allowing reasoning to be disabled → 3 min / query	Replaced by Claude Haiku 5.5 (no hidden reasoning)

6. Limitations and next steps

- Multi-document regulatory comparisons (family B) have the lowest answer precision: answers mix rules from 3–4 documents. Next: answer sections per document / regime.
- Tables without ruled lines in annual reports are parsed as text (gold-text coverage 0.72–0.77 on three reports).
- Only PDFs are indexed (other files in the zip are skipped and reported).
- Routing sends the whole card catalogue in one prompt — comfortable up to a few hundred documents.

7. API, deployment and how to test

Endpoint	Input	Output
`POST /index`	multipart: `file` (zip of documents), `collection_name`	`{"status": "success", "collection_name", "stats"}` once indexing is complete; 400 for a bad / PDF-less zip, 502 if the LLM is unavailable (existing collection untouched)
`POST /query`	JSON: `{"collection_name", "query"}`	`{"answer", "chunks": [{document, page, section, grade, text}], "trace": [{step_name, description, input, output}], "metadata": {usage, cost, latency, step_seconds, langfuse_trace_id}}`
`GET /health`	—	`{"status": "ok"}`

```
docker build -t smart-kb . && docker run -d --env-file .env -p 8000:8000 -v kb-data:/data smart-kb
curl -F "file=@corpus.zip" -F "collection_name=test" http://<host>:8000/index
curl -H "Content-Type: application/json" -d '{"collection_name": "test", "query": "..."}'
http://<host>:8000/query
```

Stated ingestion time: ~15 min cold for ~7,000 pages on 8 cores (quote ~20–25 min with headroom). Stack: Python 3.12, FastAPI, PyMuPDF 1.26.5, Tesseract, fastembed (MiniLM), rank-bm25, datasketch, Anthropic SDK → OpenCode, Langfuse. Source: private GitHub repository `smart-knowledge-base`.